Student Report Card System

Danci Gabriella

Gr: 30424

Contents:

1. Assignment objective……
2. Problem analysis, Modeling, Scenario, Use Cases……...
3. Design……...……
4. Implementation…….
5. Results……..
6. Conclusions……..

1. Assignment Objective

Main objective: Design and implement a Student Report Card System with a dedicated graphical interface through which the user can login as either a teacher or student, select the operation to be performed and view the result.

Sub-objective	Description	Section addressed
Analyze and identify requirements	Identify functional and non-functional needs for the student management system.	Problem Analysis & Use Cases
Design architecture	Develop the object-oriented design of the report card system, including user interface components and backend logic.	Design
System implementation	Translate the design specifications into executable code, including classes, methods, and graphical user interface components.	Implementation
Test the implementation	Execute various testing scenarios to ensure functionality, correctness, and reliability.	Results

2. Problem analysis, Modeling, Scenario, Use Cases

- Functional requirements:
    - Perform CRUD (Create, Read, Update, Delete) operations on student grades and attendance.
    - Display the results in a user-friendly graphical interface.
- Non-functional requirements:
    - Intuitive interface
    - Clear error handling
    - Modern design
    - Responsive layout
- Use Cases:
    - Authentication:

Scenario: A user (Student or Teacher) attempts to access their personal dashboard.
Input: Username, password, and role selection.

Action: AuthManager calls the respective authenticate method from StudentRepository or TeacherRepository.
Output: If credentials match, the User is redirected to their specific dashboard; otherwise, access is denied.

- Grade Management

Scenario: A Teacher records a new grade for a student assigned to their class.
Input: Student object, Grade details (Subject, Mark, Date, and Comments).
Action: Teacher entity validates the student is assigned to them, then StudentManager persists the grade.
Output: The grade is added to the student's record, and the system automatically invokes calculateGPA to update the student's average.

- Attendance Tracking

Scenario: A Teacher records a student's daily attendance status.
Input: Student name and Attendance details.
Action: The system validates the input; StudenManager attaches the Attendance record to the student's profile.
Output: The record is saved, and the student's attendance rate is updated in their next summary report.

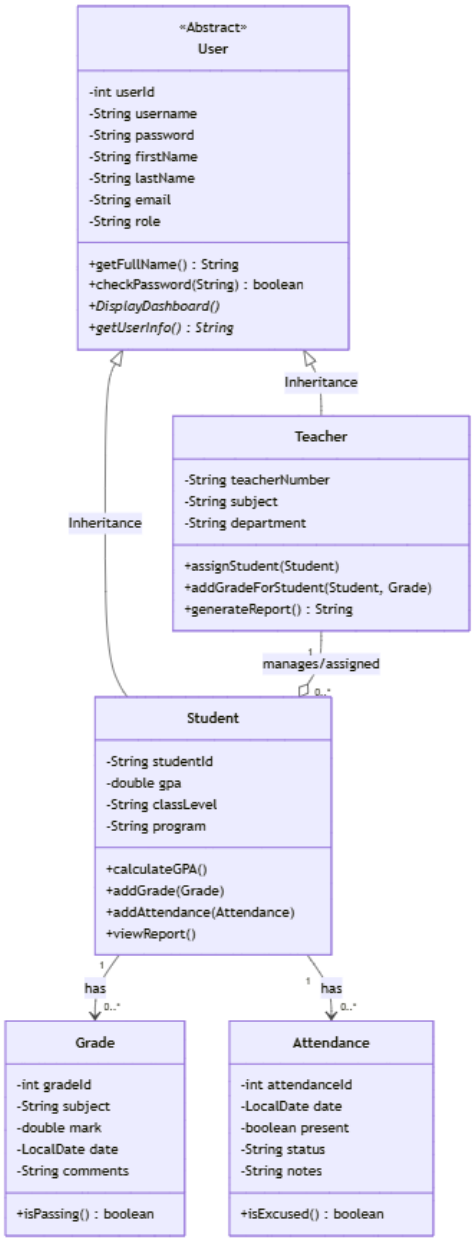
- Class performance reporting

Scenario: A Teacher generates a summary of all students assigned to their department.
Input: Teacher selects "Generate Class Report" from the dashboard.
Action: Teacher.generateReport iterates through the assigned students list, compiling individual GPAs and grades count.
Output: A "Class Summary Report" is displayed showing the Class Average GPA and student performance metrics.

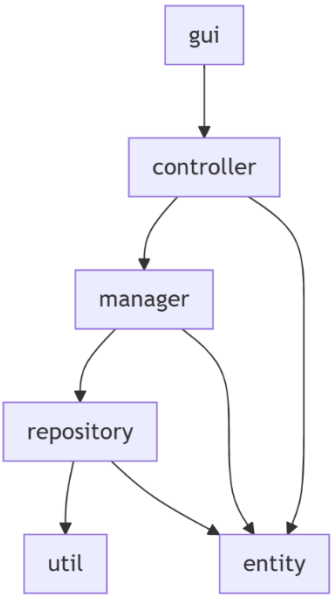
- Student report card generation

Scenario: A Student wants to view their complete academic and attendance history.
Input: Student selects "Generate Report Card" from the student dashboard.
Action: The Student entity's generateReport method aggregates all Grade and Attendance records into a formatted string.
Output: A "Student Report Card" is displayed, including the Program, Class Level, Overall GPA, and Attendance Rate percentage.

3. Design



- Class diagram



Package Diagram

4. Implementation

LoginController:

- **Fields:**
  - view: LoginView - Instance of the login interface.
  - router: Router - Handles navigation between different views.
  - auth: AuthManager - Manages authentication logic.
  - studentController: StudentController - Controller for student-specific actions.
  - teacherController: TeacherController - Controller for teacher-specific actions.
- **Important Methods:**
  - wire(): Sets up the event listener for the login button, validates inputs, and triggers the authentication process.
- **Description:** This class acts as the entry point for the application. It captures user credentials, communicates with the AuthManager to verify them, and uses the Router to redirect the user to either the Student or Teacher dashboard based on their role.

StudentController:

- **Fields:**
  - view: StudentDashboardView - The dashboard interface for students.
  - router: Router - Handles navigation.
  - current: Student - The currently logged-in student entity.
- **Important Methods:**
  - onLoggedIn(Student student): Initializes the dashboard with student data and wires buttons for viewing grades, attendance, and generating reports.
  - wireCommon(): Handles universal actions like the logout functionality.
- **Description:** This class manages the student's interaction with their dashboard. It binds student data to the UI and triggers dialogs for academic information like report cards and attendance records.

TeacherController:

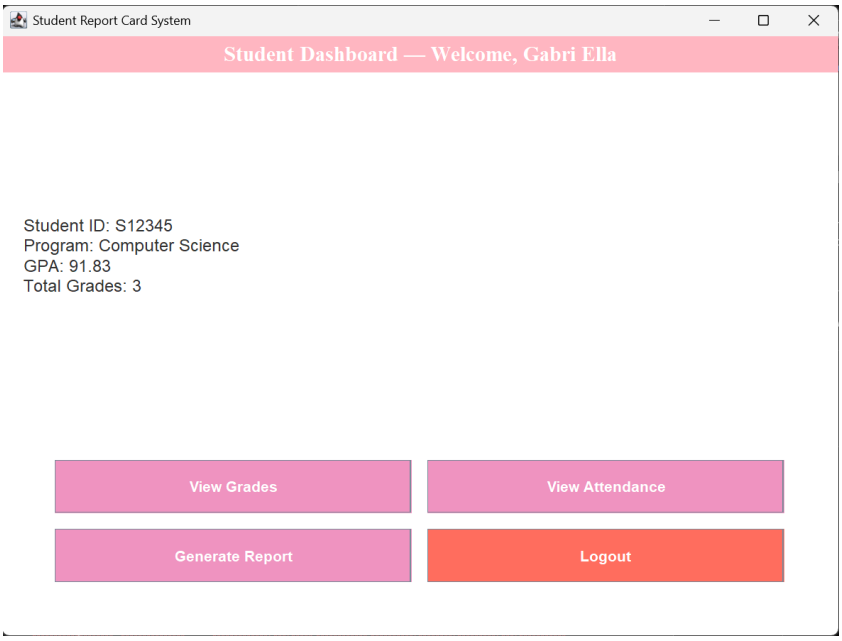
- **Fields:**
  - view: TeacherDashboardView - The dashboard interface for teachers.
  - router: Router - Handles navigation.
  - studentManager: StudentManager - Handles data operations for students (adding grades/attendance).
  - teacherManager: TeacherManager - Manages teacher data and profile reloading.
  - current: Teacher - The currently logged-in teacher entity.
- **Important Methods:**
  - onLoggedIn(Teacher teacher): Sets up the teacher's dashboard, enabling features to view students and add academic records.
  - showClassReport(): Generates and displays a comprehensive report for the teacher's classes.
  - reloadTeacher(): Synchronizes the current teacher's data with the database.
- **Description:** This class coordinates the administrative tasks of a teacher. It allows the teacher to manage student performance by adding grades and attendance, and provides tools to generate high-level class reports.

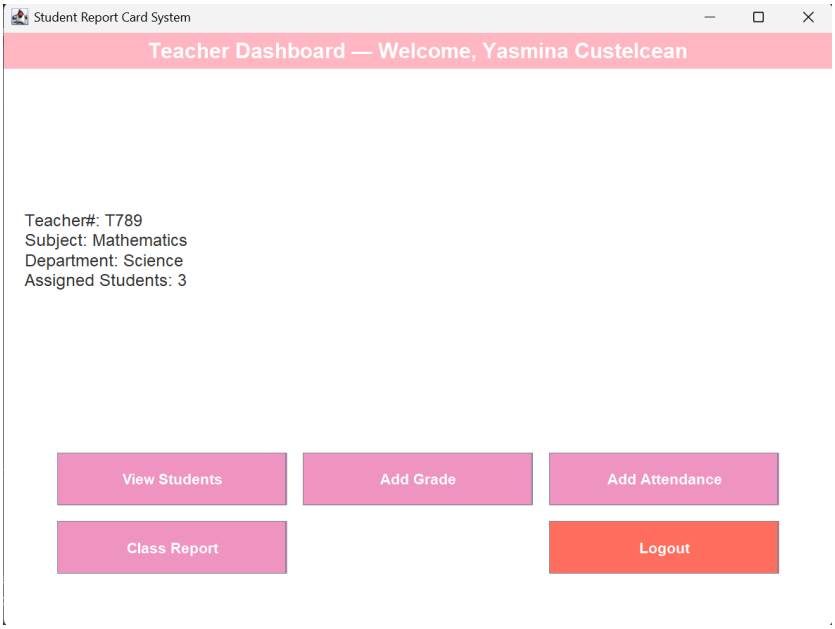
Graphical User Interface (GUI) Implementation

The graphical user interface (GUI) is implemented using several specialized view classes and a central management system. These classes provide a user-friendly environment for both students and teachers to interact with the report card system.

- **MainWindow:**
  - Main Frame: A JFrame serves as the primary window for the application.
  - Main Panel: Uses a CardLayout to facilitate smooth transitions between the login screen and the dashboards.
- **Router:**
  - Fields: Maintains a Map<String, JComponent> to track registered application screens.
  - Description: This utility class manages navigation. It allows different controllers to switch the visible content of the MainWindow by referencing specific screen names (e.g., "LOGIN", "STUDENT").
- **LoginView:**
  - Fields: Contains JTextField for username, JPasswordField for secure entry, and a ButtonGroup of JRadioButton components for role selection.
  - Styling: Utilizes a GridBagLayout for precise component positioning and features a "Times New Roman" title styled with a soft pink color (RGB: 255, 182, 193).
  - Description: Provides the entry interface where users select their role and provide credentials to access the system.

- **DashboardView (Student & Teacher):**
  - Header Panel: Displays a welcome message and user-specific details (like Student ID or Subject) in a highlighted header.
  - Action Buttons: Organized in a GridLayout, these buttons are color-coded: primary actions (like "Add Grade" or "View Attendance") are styled in pink, while "Logout" is styled in danger-red for visual clarity.
  - Dialogs: Includes static methods to generate pop-up dialogs (using JOptionPane and JScrollPane) for viewing detailed records or filling out forms to add new academic data.

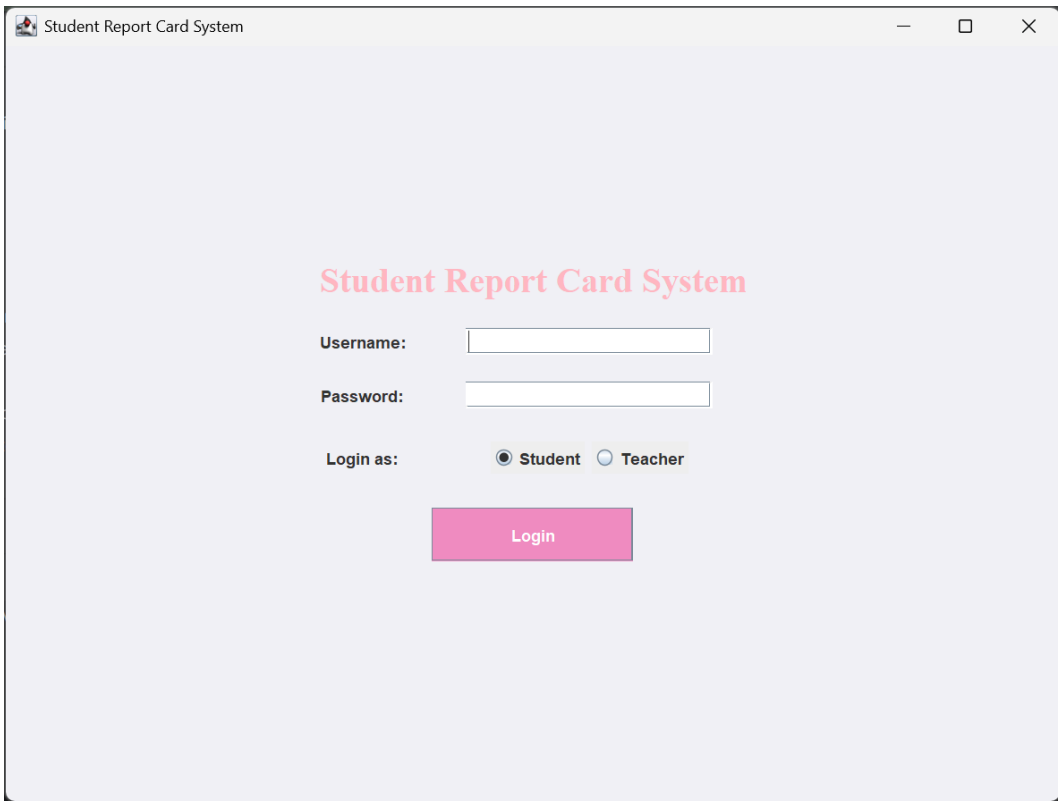




- **Action Listeners:**

- Description: Listeners are attached to the buttons within these views to handle user interactions, such as triggering the login process, opening data dialogs, and navigating between the application's different functional areas.

The GUI elements are styled using colors and fonts to enhance readability and visual appeal. User input is validated and processed to perform the desired actions.



5. Results

The Student Report Card project underwent rigorous testing to ensure its functionality and reliability. Testing scenarios were devised for the connection to the database, the student and teacher repositories to make sure the data was stored properly.

6. Conclusions

The implementation of the Student Report Card System successfully demonstrates a robust application of the Model-View-Controller (MVC) design pattern to manage academic data. By decoupling the graphical user interface from the underlying business logic, the system ensures that user interactions—whether by students viewing their GPA or teachers recording new grades—are handled efficiently and securely.

Key takeaways from this assignment include crafting Java Swing-based graphical user interfaces (GUIs), and implementing effective user input validation and error handling mechanisms.

Potential future enhancements would include adding the possibility of a teacher to add a new student to their class and refining the GUI with enhanced features and customization options.